BORDERLESS
<CODING>

Rust for Web3 development

Part 02 · Intro to Rust for Web3 with Anchor

Create a simple on-chain account from scratch

Define: who pays, where data is, required account size

> Build one instruction end-to-end with Anchor

What you will learn (1/2)

1. **`#[program]`** handler: where every instruction is registered
2. **Handlers**: a function that is called when an instruction is executed
3. **`Context<T>`**: typed bundle of accounts for one instruction

What you will learn (2/2)

4. **Account structs**: define the accounts that are needed for the instruction
5. **System Program**: creating accounts on-chain to persist data (bytes)
6. **`init` + `space`**: who pays, how many bytes (discriminator + your data)

From Part 01 to Part 02

Part 01 recap (what we build on)

- **Programs** are stateless
  - **Accounts** hold **bytes** on-chain
- **Transactions** call **instructions**
  - providing an explicit **account list**
- **Anchor**:
  - gave you **`#[program]`**
  - **`declare_id!`**
  - and an empty struct for `hello`

> ***Part 02:*** *now we* ***create*** *an account and* ***write*** *its fields safely*

The `#[program]` module

`#[program]` — entry point

A **crate** is one Rust package/project (the program folder with its own `Cargo.toml`)

- One **`#[program]`** module per crate:
  - Each **`pub fn`** inside the module is a **callable instruction** (name + arguments + accounts)
  - **`use super::*;`** brings account structs and types into scope

> *Mental model:* ***one module = your on-chain API surface***

Example: `lib.rs` (program + handler)

```rust
#[program]
pub mod my_program {
    use super::*;

    pub fn initialize(ctx: Context<Initialize>, data: u64) -> Result<()> {
        // ...
        Ok(())
    }
}
```

Same `#[program]` block (1/3) — macro + module

```rust
// Anchor turns #[program] into the on-chain
// router + IDL wiring for each pub fn.

#[program]
pub mod my_program {
    use super::*;

    pub fn initialize(ctx: Context<Initialize>, data: u64) -> Result<()> {
        // ...
        Ok(())
    }
}
```

Same block (2/3) — `use super::*`

```rust
#[program]
pub mod my_program {

    // Account structs like `Initialize`
    // live in the parent module — import them here.

    use super::*; // --> brings account structs and types into scope

    pub fn initialize(ctx: Context<Initialize>, data: u64) -> Result<()> {
        // ...
        Ok(())
    }
}
```

Same block (3/3) — handler signature

```rust
#[program]
pub mod my_program {
    use super::*;

    // `ctx` = typed, validated accounts;
    // `data` = instruction arg from the transaction.
    // Return Ok(()) on success; Err(...) reverts the whole transaction.
    // Analogy: HTTP handler — but state is only what you touch through `ctx`.
    // Note: Err(...) reverts the whole transaction.

    pub fn initialize(ctx: Context<Initialize>, data: u64) -> Result<()> {
        // ... e.g. ctx.accounts.new_account.data = data;
        Ok(()) // --> ok return
        // Example error path: return Err(ProgramError::InvalidArgument.into());
        // --> reverts the whole transaction
    }
}
```

Instruction handlers

Handlers — the on-chain “endpoints”

- Signature pattern:

```
pub fn name(ctx: Context<AccountsStruct>, …args) -> Result<()>
```

- **`ctx.accounts`** — reach validated accounts; **`args`**
  - Instruction data is BORSH serialized (binary format)
- **Return `Ok(())`** on success;
  - **`err!(MyError::…)`** or **`Err(…)`** rolls the **whole transaction** back

> *Analogy: **HTTP handler** — but state is **only** via **`ctx`***

BORSH example with a simple counter

```rust
use borsh::{BorshDeserialize, BorshSerialize};

#[derive(BorshSerialize, BorshDeserialize, Debug)]
struct Counter {
    value: u64,
}

let counter = Counter { value: 42 };
let bytes = counter.try_to_vec()?;                  // serialize -> bytes
let decoded = Counter::try_from_slice(&bytes)?;     // deserialize -> struct
assert_eq!(decoded.value, 42);
```

- On-chain accounts store **bytes**, so your struct is encoded/decoded with BORSH

BORSH bytes for a counter (u64, little-endian)

Value	BORSH bytes (hex)
0	00 00 00 00 00 00 00 00
1	01 00 00 00 00 00 00 00
2	02 00 00 00 00 00 00 00
255	ff 00 00 00 00 00 00 00
256	00 01 00 00 00 00 00 00

Example discriminator for `Counter`

```
sha256("account:Counter")[0..8] = "ff b0 04 f5 bc fd 7c 19"
```

For `struct Counter { value: u64 }`, these 8 bytes are the full account payload (before Anchor discriminator).

Full bytes for `Counter { value: 1 }`: `ff b0 04 f5 bc fd 7c 19 | 01 00 00 00 00 00 00 00`

`Context` and account bundles

`Context<T>` — typed account list

- `T` is your `#[derive(Accounts)]` struct
  - **compile-time** list of what this instruction needs
- `ctx.accounts`
  - fields of `T`
  - `ctx.program_id`
  - `ctx.bumps` (for PDAs later)

> *Anchor checks **constraints before** your handler body runs — wrong combo → **instruction error***

```
#[derive(Accounts)]
```

Account structs with `#[derive(Accounts)]`

- `pub struct Initialize<'info> { … }`
  - List `AccountInfo` wrappers with **roles**
- `'info` lifetime ties handles to the transaction's **account metas**
- Each field has **attributes**:
  - `#[account(mut)]`
  - `init`
  - `has_one = …`
  - etc.

Account structs with #[derive(Accounts)]

- Order and types **must** match what the **client** passes
  - Anchor encodes that in the IDL
  - Example:

```
pub fn set_value(ctx, value: u64) -> Result<()> {
  ctx.accounts.counter.value = value;
  Ok(())
}
```

```
program.methods.setValue(new BN(5))
  .accounts({
    counter: counterPubkey // --> counter is the account name from the IDL
  })
  .rpc();
```

Example IDL for `setValue`

IDL is the contract between Rust and TypeScript. Similar to an API contract. In EVM blockchains, the **ABI** is the contract between smart contract and client.

```
{
  "name": "setValue",
  "discriminator": [/* 8 bytes */],
  "accounts": [
    {
      "name": "counter",
      "writable": true
    }
  ],
  "args": [
    {
      "name": "value",
      "type": "u64"
    }
  ]
}
```

Common account constraints (cheat sheet)

Attribute / type	Role
`Signer<'info>`	Must have signed; often **`mut`** if it pays rent or fees
`#[account(mut)]`	Account data may be **written** in this instruction
`#[account(init, payer = …, space = …)]`	Create **new** account; **payer** funds rent; **space** = bytes
`Program<'info, System>`	Reference to **System Program** CPI for create / allocate
`Account<'info, MyData>`	Account whose owner is this program; data = **`MyData`** layout

`Signer<'info>` example

```rust
#[derive(Accounts)]
pub struct Update<'info> {
    pub authority: Signer<'info>,
}
```

- Constraint effect: transaction must include a valid signature for `authority`
- Typical use: authorization / payer identity
- Fails if the account is not a signer

`#[account(mut)]` example

```rust
#[derive(Accounts)]
pub struct Update<'info> {
    #[account(mut)]
    pub counter: Account<'info, Counter>,
}
```

- Constraint effect: marks account writable in account metas
- Without **`mut`**, trying to write to **`counter`** data fails at runtime

`init, payer, space` example

```rust
#[derive(Accounts)]
pub struct Initialize<'info> {
    #[account(init, payer = authority, space = 8 + 8)]
    pub counter: Account<'info, Counter>,
    #[account(mut)]
    pub authority: Signer<'info>,
    pub system_program: Program<'info, System>,
}
```

- `init`: create new account owned by this program
- `payer` + `space`: funds rent-exempt bytes for discriminator + data
- It runs when this specific instruction is called (not at deploy time).
- Program can run without calling this instruction, but it won't have the account created.

`Program<'info, System>` example

```rust
#[derive(Accounts)]
pub struct Initialize<'info> {
    pub system_program: Program<'info, System>,
}
```

- Constraint effect: enforces this account is the real System Program id
- Required for account creation/allocation CPI flows
- It can fail if the account is not the real System Program id (programmatic error).

`Account<'info, MyData>` example

```rust
#[account]
pub struct Counter {
    pub value: u64,
}

#[derive(Accounts)]
pub struct Update<'info> {
    pub counter: Account<'info, Counter>,
}
```

- Constraint effect: owner + data layout must match **`Counter`**
- Anchor deserializes bytes into typed Rust struct for safe access
- If the account is not owned by the program, the instruction will fail.
- If the account data layout does not match the **`Counter`** struct, the instruction will fail.

System Program & creating accounts

Why `system_program: Program<'info, System>`?

- **`init`** creates accounts through the **System Program** (via **CPI**).
- Include **`system_program`** and a **`payer`** with enough SOL.
- If **`init`** fails on devnet, check **SOL** and **`space`**.

`init` and `space`

`#[account(init, payer = …, space = …)]`

- **`init`** creates a program-owned account.
- **`payer`** (mutable signer) funds it; **`space`** = 8 + serialized struct size.
- After init, set fields (e.g. `ctx.accounts.my_account.data = value;`).

One instruction, end to end (diagram)

flowchart LR C[Client / test] -->|tx + accounts| H[Handler] H --> X[Context validates] X --> A[(New account)] H -->|writes| A

Example: `lib.rs` (program + handler)

```rust
use anchor_lang::prelude::*;

declare_id!("Fg6PaFpoGXkYsidMpWTK6W2BeZ7FEfcYkg476zPFLSn");

#[program]
pub mod example {
    use super::*;

    pub fn initialize(ctx: Context<Initialize>, data: u64) -> Result<()> {
        ctx.accounts.new_account.data = data;
        msg!("data = {}", data);
        Ok(())
    }
}
```

Link to code on GitHub

Example: `lib.rs` (accounts + data struct)

```rust
#[derive(Accounts)]
pub struct Initialize<'info> {
    #[account(mut)]
    pub signer: Signer<'info>,
    #[account(
        init,
        payer = signer,
        space = 8 + 8,
    )]
    pub new_account: Account<'info, NewAccount>,
    pub system_program: Program<'info, System>,
}

#[account]
pub struct NewAccount {
    pub data: u64,
}
```

Link to code on GitHub

Example: `client.ts` (Playground-style)

```
const tx = await pg.program.methods
  .initialize(new BN(42))
  .accounts({
    signer: pg.wallet.publicKey,
    newAccount: newAccountPubkey,
    systemProgram: SystemProgram.programId,
  })
  .rpc();

await pg.connection.confirmTransaction(tx, "confirmed");
```

Link to code on GitHub

- **`newAccountPubkey`** — a **Keypair** you generate client-side for the new PDA-less account (or use Anchor's **seeds** later)

Client mental model

- **Method + args** must match the Rust handler.
- **`.accounts({ ... })`** keys must match the accounts struct (IDL).
- Include **`SystemProgram.programId`** when using **`init`**.

Example: `anchor.test.ts`

```
describe("initialize", () => {
  it("sets data", async () => {
    const kp = Keypair.generate();
    const sig = await pg.program.methods
      .initialize(new BN(7))
      .accounts({
        signer: pg.wallet.publicKey,
        newAccount: kp.publicKey,
        systemProgram: SystemProgram.programId,
      })
      .signers([kp])
      .rpc();
    if (!sig) throw new Error("no signature");
  });
});
```

Link to code on GitHub

Debugging checklist

- `AccountNotInitialized` / `Constraint`: re-check `#[derive(Accounts)]` vs client accounts.
- `insufficient funds`: airdrop on devnet and `mut` on payer/signer.
- `space` sizing: discriminator + max serialized layout size.
- Use `msg!` for lab debugging; remove/gate in production.

Part 02 recap

- **`#[program]`** defines instructions; **`Context<T>`** carries validated accounts.
- **`#[derive(Accounts)]`** sets rules: signer, `mut`, `init`, payer, `system_program`.
- **`init`** + **`space`** create the account and cover rent.
- Clients/tests must pass all accounts from the IDL.

Next (Part 03) — Accounts & state

- Account **lifecycle:** read, update, close patterns
- **Realloc**, **seeds**, common footguns
- **Logs** and explorers as your debugger

```
state in accounts
instructions change it
constraints say who may touch what
```